

✎ 🚀 Your First LLM Call with Gemini

Day 1 — Notebook 3 of 3 | Estimated Time: 45 minutes

What You'll Learn

- How to set up the Gemini API client
- How to make your first text generation call
- How to control generation with temperature, top-k, and top-p
- How to count tokens using the API

✎ 1. Setup

First, let's install the required package and set up authentication.

🔑 **Authentication:** You don't need an API key directly. Enter the course passphrase when prompted.

```
%pip install -U -q "google-genai>=1.0.0"
```

```
import sys, os

import time

import json
import re
from google import genai
from google.genai import types
from IPython.display import Markdown

try:
    from google.colab import auth
    auth.authenticate_user()
    print("✅ Authenticated via Colab")
except ImportError:
    print("✅ Using local credentials (gcloud auth)")

client = genai.Client(
    vertexai=True,
    project="chanakya-476010",
    location="us-central1",
)
MODEL_ID = "gemini-2.5-flash"
print("✅ Ready!")
```

```
/usr/local/lib/python3.12/dist-packages/google/auth/_default.py:113: UserWarning: Your application has authenticated using end user
warnings.warn(_CLOUD_SDK_CREDENTIALS_WARNING)
✅ Authenticated via Colab
✅ Ready!
```

```
# We'll use this model throughout the notebook
MODEL_ID = "gemini-2.5-flash"
```

✎ 2. Your First API Call

The simplest way to use Gemini is the `generate_content` method. You pass in a prompt (string) and get back a response.

```
from IPython.display import Markdown

# Your very first LLM call!
response = client.models.generate_content(
```

```

model=MODEL_ID,
contents="Explain what artificial intelligence is in 3 sentences.",
)

Markdown(response.text)

```

/usr/local/lib/python3.12/dist-packages/google/auth/_default.py:113: UserWarning: Your application has authenticated using end user accounts on Google Cloud. Use app credentials to enable secure authentication. For more information about how to write a secure application, see the Google Cloud documentation: <https://cloud.google.com/docs/authentication/external/get-started-quickstart>

Artificial intelligence (AI) is a field of computer science dedicated to creating machines capable of performing tasks that typically require human intelligence. This involves developing systems that can learn from data, reason, solve problems, understand language, and perceive their environment. Ultimately, AI aims to enable computers to simulate and even enhance human cognitive functions.

✓ Understanding the Response Object

The response contains more than just text. Let's explore it:

```

# The response object has several useful attributes
print("Response text (first 200 chars):")
print(response.text[:200])
print()

# Token usage information
if response.usage_metadata:
    print("Token Usage:")
    print(f"  Input tokens:  {response.usage_metadata.prompt_token_count}")
    print(f"  Output tokens: {response.usage_metadata.candidates_token_count}")
    print(f"  Total tokens:  {response.usage_metadata.total_token_count}")

```

Response text (first 200 chars):
Artificial intelligence (AI) is a field of computer science dedicated to creating machines capable of performing tasks that typi

Token Usage:
Input tokens: 10
Output tokens: 64
Total tokens: 174

✓ 3. Multi-line Prompts

For longer prompts, use Python's triple-quoted strings. This is useful when you need to give the model detailed instructions.

```

prompt = """
You are a helpful science teacher.
Explain photosynthesis to a 10-year-old student.
Keep your explanation under 100 words.
Use simple language and a fun analogy.
"""

response = client.models.generate_content(
    model=MODEL_ID,
    contents=prompt,
)

Markdown(response.text)

```

Hey there! Imagine a plant is like a super-chef with its own special kitchen. It uses sunlight for energy, drinks water, and breathes in a gas called carbon dioxide. With these ingredients, the plant magically *cooks* up its own sugary food to grow big and strong! As a fantastic bonus, it releases oxygen, which we need to breathe. So, plants are amazing food makers for the world!

4. Controlling Generation: Temperature

Temperature controls the randomness of the model's output:

- `temperature=0` → Deterministic, always picks the most likely token
- `temperature=1` → Balanced (default)
- `temperature=2` → Very creative/random

Let's see the difference:

```
from google.genai import types

prompt = "Write a one-sentence description of the moon."

# Low temperature (more deterministic)
print("🧊 Temperature = 0 (deterministic)")
print("=" * 50)
for i in range(3):
    response = client.models.generate_content(
        model=MODEL_ID,
        contents=prompt,
        config=types.GenerateContentConfig(temperature=0),
    )
    print(f"Run {i+1}: {response.text.strip()}")

print()

# High temperature (more creative)
print("🔥 Temperature = 2 (creative)")
print("=" * 50)
for i in range(3):
    response = client.models.generate_content(
        model=MODEL_ID,
        contents=prompt,
        config=types.GenerateContentConfig(temperature=2),
    )
    print(f"Run {i+1}: {response.text.strip()}")
```

🧊 Temperature = 0 (deterministic)

```
=====
Run 1: Earth's only natural satellite, the Moon is a barren, rocky body that orbits our planet and reflects the sun's light, i
Run 2: Earth's only natural satellite, the Moon is a barren, rocky celestial body that orbits our planet and reflects the sun'
Run 3: Earth's only natural satellite, the Moon is a barren, rocky celestial body that reflects sunlight and influences our pl
```

🔥 Temperature = 2 (creative)

```
=====
Run 1: Earth's only natural satellite, the Moon is a barren, rocky sphere that constantly orbits our planet and shines by refl
Run 2: Earth's only natural satellite, the Moon is a cold, rocky, cratered orb that reflects the sun's light across our night
Run 3: Earth's only natural satellite, a barren, rocky celestial body reflecting sunlight as it orbits our planet.
```

💡 **Notice:** With `temperature=0`, the outputs are nearly identical. With `temperature=2`, each run produces different, more creative phrasing.

5. Controlling Generation: Top-K and Top-P

These parameters further control which tokens the model considers:

- **Top-K:** Only consider the K most likely tokens at each step
 - `top_k=1` → Only the single most likely token (very deterministic)
 - `top_k=40` → Consider top 40 tokens (more variety)
- **Top-P (nucleus sampling):** Consider tokens until cumulative probability reaches P
 - `top_p=0.1` → Only the most confident predictions
 - `top_p=0.95` → Consider a wider range of tokens

```

prompt = "List 5 creative names for a coffee shop."

# Restrictive: low top-k, low top-p
print("🔒 Restrictive (top_k=1, top_p=0.1)")
print("=" * 50)
response = client.models.generate_content(
    model=MODEL_ID,
    contents=prompt,
    config=types.GenerateContentConfig(top_k=1, top_p=0.1),
)
print(response.text)

print()

# Open: high top-k, high top-p, high temperature
print("🌈 Open (top_k=10, top_p=0.99, temperature=1.5)")
print("=" * 50)
response = client.models.generate_content(
    model=MODEL_ID,
    contents=prompt,
    config=types.GenerateContentConfig(top_k=10, top_p=0.99, temperature=1.5),
)
print(response.text)

```

```

🔒 Restrictive (top_k=1, top_p=0.1)
=====
Here are 5 creative names for a coffee shop, playing with different vibes:

1.  **The Daily Drip:** A playful twist on "the daily grind," suggesting a consistent and comforting coffee ritual.
2.  **Aroma Alchemy:** Evokes the magical transformation of beans into a delicious brew, highlighting the sensory experience and
3.  **The Golden Hour Cafe:** Suggests a warm, inviting atmosphere, like the beautiful light of dawn or dusk, perfect for a rela
4.  **The Roaming Bean:** Implies a journey of discovery, perhaps featuring different coffee origins or a sense of adventure and
5.  **Chapter & Verse Coffee:** A literary-inspired name, perfect for a cozy spot where people can read, write, or have deep con

🌈 Open (top_k=10, top_p=0.99, temperature=1.5)
=====
Here are 5 creative names for a coffee shop, aiming for different vibes:

1.  **Aura Brew:** (Evokes a feeling, an atmosphere, and a place for making coffee.)
2.  **Grounds for Joy:** (A clever play on words, suggesting both coffee grounds and reasons to be happy.)
3.  **The Daily Spark:** (Short, energetic, suggests coffee provides a daily kick-start and inspiration.)
4.  **Whispering Grounds:** (Conveys a cozy, intimate atmosphere, perfect for quiet conversations over coffee.)
5.  **Bean Alchemy:** (Highlights the craft, transformation, and almost magical quality of turning beans into delicious coffee.)

```

6. Controlling Output Length: Max Output Tokens

You can limit how many tokens the model generates with `max_output_tokens`:

```

prompt = "Tell me the history of the internet."

# Short response
print("📄 Short (max_output_tokens=50)")
print("=" * 50)
response = client.models.generate_content(
    model=MODEL_ID,
    contents=prompt,
    config=types.GenerateContentConfig(max_output_tokens=50),
)
print(response.text)
print(f"\n(Output tokens used: {response.usage_metadata.candidates_token_count})")

print()

# Longer response
print("📄 Longer (max_output_tokens=300)")
print("=" * 50)
response = client.models.generate_content(
    model=MODEL_ID,
    contents=prompt,
    config=types.GenerateContentConfig(max_output_tokens=300),
)
print(response.text)
print(f"\n(Output tokens used: {response.usage_metadata.candidates_token_count})")

```

```

 Short (max_output_tokens=50)
=====
The internet

(Output tokens used: 2)

 Longer (max_output_tokens=300)
=====
The history of the internet is a fascinating journey from a Cold

(Output tokens used: 12)

```

7. Counting Tokens

You can count tokens before sending a request — useful for cost estimation and checking context window limits:

```

# Count tokens for different texts
texts = [
    "Hello!",
    "The quick brown fox jumps over the lazy dog.",
    "Explain quantum computing in simple terms, covering superposition, entanglement, and quantum gates.",
]

print("Token Counting")
print("=" * 60)
for text in texts:
    token_count = client.models.count_tokens(
        model=MODEL_ID,
        contents=text,
    )
    words = len(text.split())
    print(f"\nText: \"{text[:60]}{'...' if len(text) > 60 else ''}\"")
    print(f"  Words: {words}")
    print(f"  Tokens: {token_count.total_tokens}")
    print(f"  Ratio: {token_count.total_tokens / words:.2f} tokens/word")

```

```

Token Counting
=====

Text: "Hello!"
Words: 1
Tokens: 2
Ratio: 2.00 tokens/word

Text: "The quick brown fox jumps over the lazy dog."
Words: 9
Tokens: 10
Ratio: 1.11 tokens/word

Text: "Explain quantum computing in simple terms, covering superpos..."
Words: 12
Tokens: 16
Ratio: 1.33 tokens/word

```

8. Listing Available Models

Let's see what Gemini models are available to us:

```

# List available models
print("Available Gemini Models")
print("=" * 70)
for model in client.models.list():
    if "gemini" in model.name.lower():
        print(f"  {model.name}")

```

Available Gemini Models

=====

```

ClientError                                Traceback (most recent call last)
/tmp/ipykernel_19522/4036597718.py in <cell line: 0>()
      2 print("Available Gemini Models")
      3 print("=" * 70)
----> 4 for model in client.models.list():
      5     if "gemini" in model.name.lower():
      6         print(f" {model.name}")

```

13 frames

```

/usr/local/lib/python3.12/dist-packages/google/genai/errors.py in raise_error(cls, status_code, response_json, response)
    182     """
    183     if 400 <= status_code < 500:
--> 184         raise ClientError(status_code, response_json, response)
    185     elif 500 <= status_code < 600:
    186         raise ServerError(status_code, response_json, response)

```

```

ClientError: 403 PERMISSION_DENIED. {'error': {'code': 403, 'message': 'Your application is authenticating by using local Application Default Credentials. The aiplatform.googleapis.com API requires a quota project, which is not set by default. To learn how to set your quota project, see https://cloud.google.com/docs/authentication/adc-troubleshooting/user-creds .', 'status': 'PERMISSION_DENIED', 'details': [{'@type': 'type.googleapis.com/google.rpc.ErrorInfo', 'reason': 'SERVICE_DISABLED', 'domain': 'googleapis.com', 'metadata': {'service': 'aiplatform.googleapis.com', 'consumer': 'projects/764086051850'}}], {'@type': 'type.googleapis.com/google.rpc.LocalizedMessage', 'locale': 'en-US', 'message': 'Your application is authenticating by using local Application Default Credentials. The aiplatform.googleapis.com API requires a quota project, which is not set by default. To learn how to set your quota project, see https://cloud.google.com/docs/authentication/adc-troubleshooting/user-creds .'}}}]

```

Next steps: [Explain error](#)

✎ 🏠 Exercise 1: Your Turn — Ask Gemini Anything

Write a prompt of your choice and send it to Gemini. Try different types of tasks:

- Ask a factual question
- Ask it to write something creative
- Ask it to explain a concept

```

# Exercise 1: Write your own prompt!
# TODO: Replace the prompt below with your own

my_prompt = "Can you give a list of Alkaline earth metals with a 4-5 line description of each of them"

response = client.models.generate_content(
    model=MODEL_ID,
    contents=my_prompt,
)

Markdown(response.text)

```

Here are the Alkaline Earth Metals, each with a 4-5 line description:

1. **Beryllium (Be)** Beryllium is the lightest alkaline earth metal, known for its high strength-to-weight ratio and exceptional stiffness. It's a steel-gray, relatively hard metal with a high melting point, forming a passive oxide layer that makes it corrosion-resistant. Used extensively in alloys for aerospace components, X-ray windows due to its transparency, and some nuclear applications. However, it is toxic and requires careful handling.
2. **Magnesium (Mg)** Magnesium is a silvery-white, lightweight, and relatively soft metal that tarnishes slightly in air. It readily burns with a brilliant white light, making it useful in flares and fireworks. It's a crucial component in numerous lightweight alloys, particularly for aerospace and automotive industries, enhancing strength without significant weight gain. Biologically, it's vital for many bodily functions and is the central atom in chlorophyll.
3. **Calcium (Ca)** Calcium is a soft, silvery-white metal that reacts readily with air and water, forming a whitish-gray oxide/hydroxide layer. It is the fifth most abundant element by mass in the Earth's crust and the most common alkaline earth metal. Widely known for its indispensable role in bones and teeth, it's also a key component in limestone, cement, plaster, and many other building materials.
4. **Strontium (Sr)** Strontium is a soft, silvery-white metal that rapidly tarnishes upon exposure to air, and it is highly reactive, reacting vigorously with water. It burns with a distinctive, bright red flame when ignited. This characteristic color is why strontium compounds are used extensively in fireworks and flares. Historically, it was also used in cathode ray tube (CRT) glass to block X-ray emissions.
5. **Barium (Ba)** Barium is a soft, silvery-white metal that is very reactive, rapidly oxidizing in air and reacting vigorously with water. Its compounds give a distinctive green color to flames, making them popular in fireworks. The insoluble compound barium sulfate is widely used as a "barium meal" for X-ray imaging of the digestive tract, despite other barium compounds being highly toxic. It's also used in drilling fluids as barite.
6. **Radium (Ra)** Radium is an extremely radioactive, silvery-white metal that quickly tarnishes black upon exposure to air. It is the heaviest of the alkaline earth metals and is highly reactive, reacting violently with water. Discovered by Marie Curie, it was historically used in luminous paints for watch dials and aircraft gauges. Today, its applications are extremely limited due to its intense radioactivity, mainly to specialized research and some medical treatments.

Exercise 2: Temperature Experiment

Pick a creative writing prompt (e.g., "Write a haiku about coding"). Run it 3 times each at temperatures 0, 1, and 2. Observe how the outputs change.

```
# Exercise 2: Temperature experiment
# TODO: Replace with your creative prompt

creative_prompt = "Write a 5 line explanation about Vedic Mathematics"

for temp in [0, 1, 2]:
    print(f"\n{' '*50}")
    print(f"Temperature = {temp}")
    print(f"{' '*50}")
    for run in range(3):
        response = client.models.generate_content(
            model=MODEL_ID,
            contents=creative_prompt,
            config=types.GenerateContentConfig(temperature=temp),
        )
        print(f"Run {run+1}: {response.text.strip()}")
    print()
```

```
=====
Temperature = 0
=====
Run 1: Vedic Mathematics is an ancient system of mathematical techniques.
It's derived from 16 simple Sutras (aphorisms) or word-formulae.
These methods offer incredibly fast and easy ways to solve complex problems.
It simplifies arithmetic, algebra, geometry, and more, often allowing mental calculation.
Ultimately, it enhances mental agility and makes mathematics more intuitive and enjoyable.

Run 2: Vedic Mathematics is an ancient Indian system of mental calculation.
It's based on 16 Sutras (aphorisms) and 13 Sub-Sutras, or short formulas.
These principles offer simple, elegant, and often one-line solutions to complex problems.
It aims to simplify arithmetic, algebra, geometry, and even calculus.
The system enhances speed, accuracy, and mental agility in mathematical operations.

Run 3: Vedic Mathematics is an ancient system of mathematical techniques.
It's derived from 16 simple Sutras (aphorisms) or word-formulae.
These methods offer incredibly fast and easy ways to solve complex problems.
It simplifies arithmetic, algebra, geometry, and more, often allowing mental calculation.
Ultimately, it enhances mental agility and makes mathematics more intuitive and enjoyable.

=====
Temperature = 1
=====
Run 1: Vedic Mathematics is an ancient system of mathematics derived from the Vedas.
It comprises 16 Sutras (aphorisms) and 13 Sub-Sutras.
These simple, elegant formulas allow for remarkably fast mental calculation.
It simplifies complex arithmetic operations like multiplication, division, and squaring.
The system fosters a deeper, intuitive understanding of numbers and problem-solving.

Run 2: Vedic Mathematics is an ancient Indian system of rapid mental calculation.
It's based on 16 simple Sutras (aphorisms) and 13 Sub-Sutras.
These unique formulas offer elegant and straightforward ways to solve complex problems.
They simplify arithmetic, algebra, and even some geometry.
The system promotes speed, accuracy, and a deeper understanding of numbers.

Run 3: Vedic Mathematics is an ancient Indian system based on 16 insightful Sutras (aphorisms).
It provides unique, intuitive methods for rapid mental calculation.
These elegant techniques simplify complex arithmetic, algebra, and even geometry.
The system aims to solve problems swiftly, often in a single line.
It fosters speed, accuracy, and a deeper, more creative understanding of mathematics.

=====
Temperature = 2
=====
Run 1: Vedic Mathematics is an ancient Indian system of rapid mental calculation and simplified arithmetic.
It is based on 16 concise Sanskrit formulas, known as Sutras, and 13 Upasutras.
These principles offer unique and often quicker methods for operations like multiplication, division, squaring, and more.
It encourages intuitive thinking and creative problem-solving, enhancing numerical agility.
Ultimately, it aims to make mathematics faster, easier, and more enjoyable to learn and practice.

Run 2: Vedic Mathematics is an ancient system of mental calculation rediscovered from Sanskrit Sutras.
It offers incredibly simple, fast, and intuitive methods for solving complex mathematical problems.
Based on 16 main Sutras (formulae) and 13 Sub-Sutras, it simplifies arithmetic, algebra, and geometry.
```

Its unique techniques often allow for single-line solutions, dramatically reducing calculation time.

✎ Exercise 3: Build a Simple Q&A Function

Create a reusable function that takes a question and returns Gemini's answer. Include token counting.

```
# Exercise 3: Build a Q&A function

def ask_gemini(question: str, temperature: float = 1.0) -> str:
    """
    Send a question to Gemini and return the response.
    Also prints token usage information.

    Args:
        question: The question to ask
        temperature: Controls randomness (0-2)

    Returns:
        The model's response text
    """
    # TODO: Complete this function
    # 1. Call client.models.generate_content with the question and temperature
    response = client.models.generate_content(
        model=MODEL_ID,
        contents=question,
        config=types.GenerateContentConfig(temperature=temperature),
    )
    # 2. Print the token usage (input, output, total)
    if response.usage_metadata:
        usage = response.usage_metadata
        print(f" Input tokens:    {usage.prompt_token_count}")
        print(f" Output tokens:    {usage.candidates_token_count}")
        print(f" Thinking tokens: {usage.thoughts_token_count}")

        # THIS is the likely culprit for the 1,324 gap:
        cached = getattr(usage, 'cached_content_token_count', 0)
        if cached == None:
            cached=0
        print(f" Cached tokens:    {cached}")

        print(f" ---")
        print(f" Calculated Sum:    {usage.prompt_token_count + usage.candidates_token_count + usage.thoughts_token_count + cached}")
        print(f" Reported Total:    {usage.total_token_count}")

    # 3. Return the response text
    return(response.text)
pass # Replace with your implementation

# Test your function
answer = ask_gemini("Summarize the poem Ozymandias and can you give me an interesting reason why it is placed before the Poem R:
if answer:
    display(Markdown(answer))
```



```

output tokens: 1116
Thinking tokens: 1429
Cached tokens: 0
---
Calculated Sum: 2574
Reported Total: 2574

```

Okay, let's break down "Ozymandias" and explore a compelling reason for its potential placement before "The Rime of the Ancient Mariner."

Summary of Percy Bysshe Shelley's "Ozymandias"

"Ozymandias" is a sonnet that tells of a traveler who encounters the ruins of a colossal statue in the desert. The statue, though shattered, depicts a king with a "frown, and wrinkled lip, and sneer of cold command." On its pedestal, an inscription boasts:

"My name is Ozymandias, King of Kings: Look on my works, ye Mighty, and despair!"

However, the poem then starkly contrasts this declaration of eternal power with the desolate reality: "Nothing beside remains. Round the decay / Of that colossal Wreck, boundless and bare / The lone and level sands stretch far away."

In essence: The poem is a powerful meditation on the transient nature of human power, ambition, and earthly achievements. It illustrates how even the grandest monuments and most boastful rulers are ultimately insignificant against the relentless, indifferent march of time and the vastness of nature. All that remains of Ozymandias's "works" is a shattered face and a boastful inscription swallowed by an empty desert.

An Interesting Reason for its Placement Before "The Rime of the Ancient Mariner"

While poetic anthologies or collections might arrange poems in various ways (chronologically, thematically, by author, etc.), an "interesting" or *thematic* reason to place "Ozymandias" directly before Samuel Taylor Coleridge's "The Rime of the Ancient Mariner" is that it serves as a **profound philosophical overture about the ultimate fate of human hubris when confronted by forces beyond its control, setting the stage for the Mariner's moral and spiritual confrontation with nature's power.**

Here's why this works:

1. The Folly of Hubris (Passive vs. Active):

- **Ozymandias:** Presents a *passive* consequence of hubris. The king's immense pride and claim to absolute power are simply eroded and forgotten by the indifferent forces of time and nature. His punishment is the complete annihilation of his legacy, leaving only a fragmented, mocking ruin. It's a visual, silent warning.
- **The Ancient Mariner:** Explores an *active* consequence of hubris. The Mariner's senseless killing of the Albatross is an act of disrespect and transgression against nature. This hubris directly invokes a supernatural, active retribution from nature itself, leading to immediate suffering, death, and a prolonged spiritual agony. It's an auditory, active warning.
- **The Link:** "Ozymandias" shows us the *end point* – the inevitable decay of human glory when pitted against nature's scale. "The Ancient Mariner" then delves into the *process* – how an act of human arrogance can actively provoke nature's wrath and lead to profound moral and spiritual consequences long before time has its final say. One is about the *vanity of earthly power*, the other about the *moral responsibility towards the natural world*.

2. The Indifference and Power of Nature:

- **Ozymandias:** Nature (the "lone and level sands") is depicted as vast, enduring, and utterly indifferent to human endeavor. It simply *outlasts* and *swallows* all human attempts at permanence.
- **The Ancient Mariner:** Nature is not just indifferent; it is animate, vengeful, and imbued with supernatural power. It actively punishes transgression, orchestrating the suffering of the crew and the Mariner through supernatural phenomena.
- **The Link:** "Ozymandias" provides a stark visual of nature's *ultimate* victory over man's pride. "The Ancient Mariner" then takes this concept further, showing nature's *immediate* and *active* capacity for retribution when disrespected, thereby establishing a cosmic scale for the moral stakes of the Mariner's journey.

3. A Shared Sense of Isolation and Desolation:

- **Ozymandias:** The "colossal Wreck" stands "boundless and bare" in a vast, empty desert – an image of profound desolation and isolation.
- **The Ancient Mariner:** The ship is becalmed and isolated in the vast, stagnant ocean ("Water, water, everywhere, / Nor any drop to drink"). The Mariner becomes profoundly isolated, first among his dying crew and then as a cursed wanderer.
- **The Link:** Both poems use images of extreme isolation in vast, unforgiving natural settings to highlight the precariousness of human existence and the profound consequences of actions (or lack thereof) in such environments.

By placing "Ozymandias" first, an editor could be setting up a grand, overarching theme: that humanity's place in the universe is fleeting, and that our pride and actions have profound, often devastating, consequences when confronted by the immense, enduring forces of the natural world, whether through slow decay or swift, supernatural retribution. "Ozymandias" is a stark, philosophical preamble that establishes the immense stage upon which the Mariner's harrowing moral drama will then unfold.

✎ Exercise 4: Token Cost Calculator

Build a function that estimates the cost of an API call based on token count.

```

# Exercise 4: Token cost calculator

# Approximate Gemini 2.5 Flash pricing (per 1M tokens)
INPUT_PRICE_PER_1M = 0.15 # $0.15 per 1M input tokens
OUTPUT_PRICE_PER_1M = 0.60 # $0.60 per 1M output tokens

def estimate_cost(prompt: str, expected_output_tokens: int = 500) -> dict:
    """
    Estimate the cost of an API call.

    Args:
        prompt: The input prompt
    """

```

```

    expected_output_tokens: Estimated number of output tokens

Returns:
    Dictionary with token counts and costs
    """

    response = client.models.generate_content(
        model=MODEL_ID,
        contents=prompt,
        config=types.GenerateContentConfig(temperature=0),
    )

    if response.usage_metadata:
        usage = response.usage_metadata
        print(f"    Input tokens:    {usage.prompt_token_count}")
        print(f"    Output tokens:    {usage.candidates_token_count}")
        print(f"    Reported Total:    {usage.total_token_count}")

        input_tokens=usage.prompt_token_count
        output_tokens=usage.candidates_token_count
        total_tokens=usage.total_token_count

    Input_cost=(input_tokens/1000000)*INPUT_PRICE_PER_1M
    Output_cost=(expected_output_tokens / 1_000_000) * OUTPUT_PRICE_PER_1M
    total_cost=Input_cost+Output_cost

    cost_dict={}
    cost_dict['input_tokens']=input_tokens
    cost_dict['output_tokens']=output_tokens
    cost_dict['input_cost']=Input_cost
    cost_dict['output_cost']=Output_cost
    cost_dict['total_cost']=total_cost

    return(cost_dict)

# TODO: Complete this function
# 1. Count input tokens using client.models.count_tokens()
# 2. Calculate input cost = (input_tokens / 1_000_000) * INPUT_PRICE_PER_1M
# 3. Calculate output cost = (expected_output_tokens / 1_000_000) * OUTPUT_PRICE_PER_1M
# 4. Return a dict with all the info

pass # Replace with your implementation

# Test: estimate cost for a long prompt
test_prompt = "Write a detailed 1000-word essay about the history of artificial intelligence."
result = estimate_cost(test_prompt, expected_output_tokens=1500)
if result:
    print(f"Input tokens:    {result.get('input_tokens', 'N/A')}")
    print(f"Output tokens:    {result.get('output_tokens', 'N/A')}")
    print(f"Input cost:       ${result.get('input_cost', 0):.6f}")
    print(f"Output cost:      ${result.get('output_cost', 0):.6f}")
    print(f"Total cost:       ${result.get('total_cost', 0):.6f}")

Input tokens:    18
Output tokens:    1477
Reported Total:    3189
Input tokens:    18
Output tokens:    1477
Input cost:       $0.000003
Output cost:      $0.000900
Total cost:       $0.000903

```

Key Takeaways

1. **Gemini API** is accessed through the `google-genai` SDK using `genai.Client()`
2. `generate_content()` is the core method for text generation